

```
#!/usr/bin/env python3
```

```
"""
```

```
chart_signal_bot.py
```

Reads a chart screenshot, asks a vision-capable Claude model to read price action / indicators / trend off the image, turns that into a BUY_UP / BUY_DOWN / WAIT signal, attaches stop-loss + position-size risk parameters, and (optionally) pushes the result to a Telegram chat.

IMPORTANT: This automates *reading a chart image* and applying rules you define. It does not have any special ability to predict price. Treat the vision model's read of the chart as one noisy input, not ground truth - back-test and paper-trade before risking real money.

Setup:

```
pip install anthropic requests
```

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

```
export TELEGRAM_BOT_TOKEN="..."      # optional, for alerts
```

```
export TELEGRAM_CHAT_ID="..."        # optional, for alerts
```

Usage:

```
python chart_signal_bot.py --image chart.png --account-size 1000 \
    --risk-pct 1.0 --symbol DOGE/USDT
```

```
"""
```

```
import argparse
```

```
import base64
```

```
import json
```

```
import os
```

```
import sys
```

```
from dataclasses import dataclass, asdict
```

```
from typing import Optional
```

```
try:
```

```
    import anthropic
```

```
except ImportError:
```

```
    anthropic = None
```

```
try:
```

```
    import requests
```

```
except ImportError:
```

```
    requests = None
```

```
VISION_MODEL = "claude-sonnet-4-6"
```

```
SYSTEM_PROMPT = """You are a technical-analysis assistant reading a single \
chart screenshot (candlesticks or line, possibly with indicators like \
RSI/MACD/moving averages/volume). You are NOT a financial advisor and you \
do not have live market data - work only from what is visibly in the image.
```

```
Return ONLY a JSON object, no prose, no markdown fences, with this exact \
shape:
```

```
{
    "symbol_guess": string or null,
    "trend_direction": "up" | "down" | "sideways" | "unclear",
    "trend_strength": "weak" | "moderate" | "strong",
    "key_levels": {"support": number or null, "resistance": number or null},
    "indicators_observed": [string, ...],
    "last_price_guess": number or null,
    "signal": "BUY_UP" | "BUY_DOWN" | "WAIT",
    "confidence": number,           // 0.0 - 1.0
    "reasoning": string,           // 1-3 sentences
    "invalidation_price": number or null // price that would prove this read wrong
}
```

Rules:

- If the chart is ambiguous, choppy, or you cannot make out clean price \ action, set signal to "WAIT" and confidence low - do not force a call.
 - confidence should reflect how clean/clear the setup is, not enthusiasm.
 - Only reference what's actually visible in the image.
- ```
"""
```

```
@dataclass
```

```
class RiskParams:
```

```
 account_size: float
```

```
 risk_pct: float # % of account risked per trade, e.g. 1.0
```

```
 entry_price: float
```

```
 stop_loss_price: float
```

```
 max_position_pct: float # hard cap on % of account in one trade
```

```
 def compute(self) -> dict:
```

```
 risk_amount = self.account_size * (self.risk_pct / 100.0)
```

```
 stop_distance = abs(self.entry_price - self.stop_loss_price)
```

```
 if stop_distance <= 0:
```

```
 position_size_units = 0.0
```

```
 else:
```

```
 position_size_units = risk_amount / stop_distance
```

```

 position_value = position_size_units * self.entry_price
 max_position_value = self.account_size * (self.max_position_pct / 100.0)
 capped = position_value > max_position_value
 if capped and self.entry_price > 0:
 position_value = max_position_value
 position_size_units = position_value / self.entry_price

 return {
 "risk_amount_usd": round(risk_amount, 2),
 "stop_distance": round(stop_distance, 6),
 "position_size_units": round(position_size_units, 6),
 "position_value_usd": round(position_value, 2),
 "capped_by_max_position_pct": capped,
 }

def encode_image(path: str) -> tuple[str, str]:
 ext = os.path.splitext(path)[1].lower().lstrip(".")
 media_type = {
 "png": "image/png", "jpg": "image/jpeg", "jpeg": "image/jpeg",
 "webp": "image/webp", "gif": "image/gif",
 }.get(ext, "image/png")
 with open(path, "rb") as f:
 data = base64.standard_b64encode(f.read()).decode("utf-8")
 return data, media_type

def analyze_chart(image_path: str) -> dict:
 if anthropic is None:
 sys.exit("Install the 'anthropic' package: pip install anthropic")
 client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from env
 img_data, media_type = encode_image(image_path)

 response = client.messages.create(
 model=VISION_MODEL,
 max_tokens=800,
 system=SYSTEM_PROMPT,
 messages=[{
 "role": "user",
 "content": [
 {"type": "image", "source": {"type": "base64", "media_type": media_type, "data": img_data}},
 {"type": "text", "text": "Read this chart and return the JSON obj"}
],
 }],
)

 raw = ""
 for block in response.content:
 if block.type == "text":
 raw = raw.removeprefix("```json").removeprefix("```").removesuffix("```").strip()

```

```

try:
 return json.loads(raw)
except json.JSONDecodeError:
 sys.exit(f"Model did not return valid JSON:\n{raw}")

def build_signal(analysis: dict, account_size: float, risk_pct: float,
 max_position_pct: float, stop_pct: float) -> dict:
 signal = analysis.get("signal", "WAIT")
 entry = analysis.get("last_price_guess")
 result = {"analysis": analysis, "risk": None}

 if signal in ("BUY_UP", "BUY_DOWN") and entry:
 if signal == "BUY_UP":
 stop = analysis.get("key_levels", {}).get("support") or entry * (1 -
 else:
 stop = analysis.get("key_levels", {}).get("resistance") or entry * (1

 risk = RiskParams(
 account_size=account_size, risk_pct=risk_pct,
 entry_price=entry, stop_loss_price=stop,
 max_position_pct=max_position_pct,
)
 result["risk"] = {"entry_price": entry, "stop_loss_price": round(stop, 6)
 return result

def send_telegram_alert(text: str) -> None:
 token = os.environ.get("TELEGRAM_BOT_TOKEN")
 chat_id = os.environ.get("TELEGRAM_CHAT_ID")
 if not (token and chat_id):
 return
 if requests is None:
 print("(install 'requests' to enable Telegram alerts)", file=sys.stderr)
 return
 url = f"https://api.telegram.org/bot{token}/sendMessage"
 try:
 requests.post(url, json={"chat_id": chat_id, "text": text, "parse_mode":
 except Exception as e:
 print(f"Telegram alert failed: {e}", file=sys.stderr)

def format_alert(symbol: Optional[str], result: dict) -> str:
 a = result["analysis"]
 lines = [
 f"*Signal: {a.get('signal')}* (confidence {a.get('confidence')})",
 f"Symbol: {symbol or a.get('symbol_guess') or 'unknown'}",

```

```

 f"Trend: {a.get('trend_direction')} / {a.get('trend_strength')}",
 f"Reasoning: {a.get('reasoning')}",
]
 if result.get("risk"):
 r = result["risk"]
 lines += [
 f"Entry: {r['entry_price']} Stop: {r['stop_loss_price']}",
 f"Position size: {r['position_size_units']} units (${r['position_valu
 f"Risking: ${r['risk_amount_usd']} + (" (capped by max position %) "
]
 if a.get("invalidation_price"):
 lines.append(f"Invalidation: {a['invalidation_price']}")
 return "\n".join(lines)

def main():
 p = argparse.ArgumentParser(description="Turn a chart screenshot into a risk-
 p.add_argument("--image", required=True, help="Path to chart screenshot")
 p.add_argument("--symbol", default=None, help="Symbol/ticker label, e.g. DOGE
 p.add_argument("--account-size", type=float, required=True, help="Account siz
 p.add_argument("--risk-pct", type=float, default=1.0, help="%% of account to
 p.add_argument("--max-position-pct", type=float, default=20.0, help="Hard cap
 p.add_argument("--stop-pct", type=float, default=2.0, help="Fallback stop dis
 p.add_argument("--min-confidence", type=float, default=0.55, help="Below this
 p.add_argument("--notify", action="store_true", help="Send result to Telegram
 args = p.parse_args()

 analysis = analyze_chart(args.image)

 if analysis.get("confidence", 0) < args.min_confidence and analysis.get("sign
 analysis["signal"] = "WAIT"
 analysis["reasoning"] = f"(forced WAIT: confidence below {args.min_confid

 result = build_signal(analysis, args.account_size, args.risk_pct, args.max_po
 result["symbol"] = args.symbol

 print(json.dumps(result, indent=2))

 alert_text = format_alert(args.symbol, result)
 if args.notify:
 send_telegram_alert(alert_text)

if __name__ == "__main__":
 main()

```

